

Особенности обучения глубоких нейронных сетей

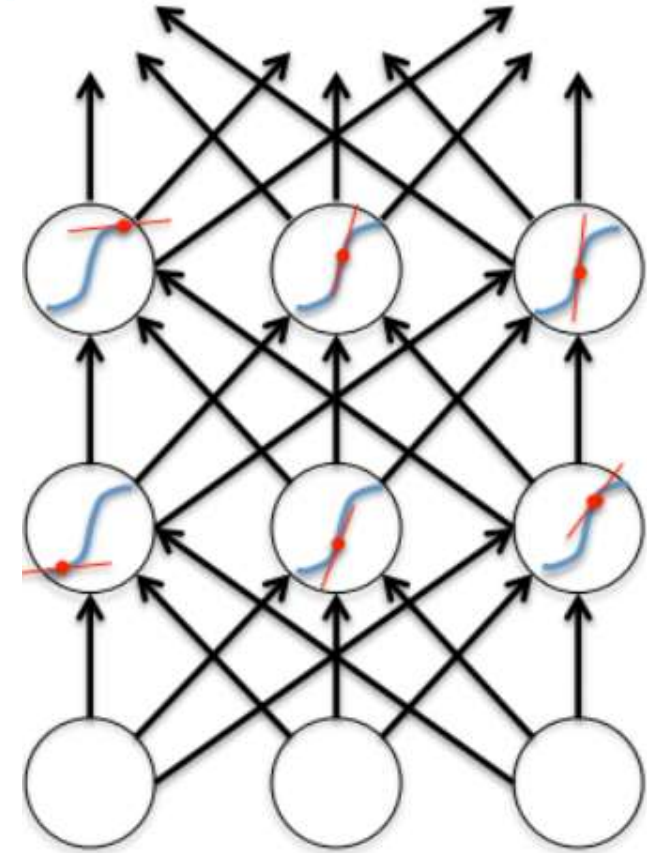
- ❑ В предыдущих разделах мы рассмотрели алгоритм обучения градиентного спуска и пример создания и обучения простого нейросетевого классификатора. Но мы имели дело с совсем небольшой сетью, имеющей всего лишь один скрытый слой.
- ❑ Что, если перед Вами стоит более сложная задача, например, обнаружение сотен типов объектов в изображениях с высоким разрешением? Вам может понадобиться обучить гораздо более глубокую сеть DNN, предположим с 10 слоями, каждый из которых содержит сотни нейронов, соединенных сотнями тысяч связей. Сделать это будет не так уж и просто.
- ❑ Во-первых, вы столкнетесь с коварной проблемой исчезновения градиентов (vanishing gradients) или связанной с ней проблемой взрывного роста градиентов (exploding gradients), которая оказывает воздействие на глубокие нейронные сети и делает низкоуровневые слои весьма трудными в обучении.
- ❑ Во-вторых, обучение такой крупной сети может стать крайне медленным.
- ❑ В-третьих, модель с миллионами параметров подвержена высокому риску переобучения на обучающем наборе.

Особенности обучения глубоких нейронных сетей

- ❑ Разберем выше упомянутые проблемы и обсудим возможные приемы для их разрешения.
- ❑ Начнем с объяснения проблемы исчезновения/взрыва градиентов и исследуем некоторые из самых популярных ее решений.
- ❑ Рассмотрим разнообразные оптимизаторы, которые могут чрезвычайно ускорить обучение крупных моделей в сравнении с обыкновенным градиентным спуском.
- ❑ Рассмотрим несколько популярных приемов регуляризации для глубоких нейронных сетей.

3.6. Проблемы исчезновения и взрывного роста градиентов. Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

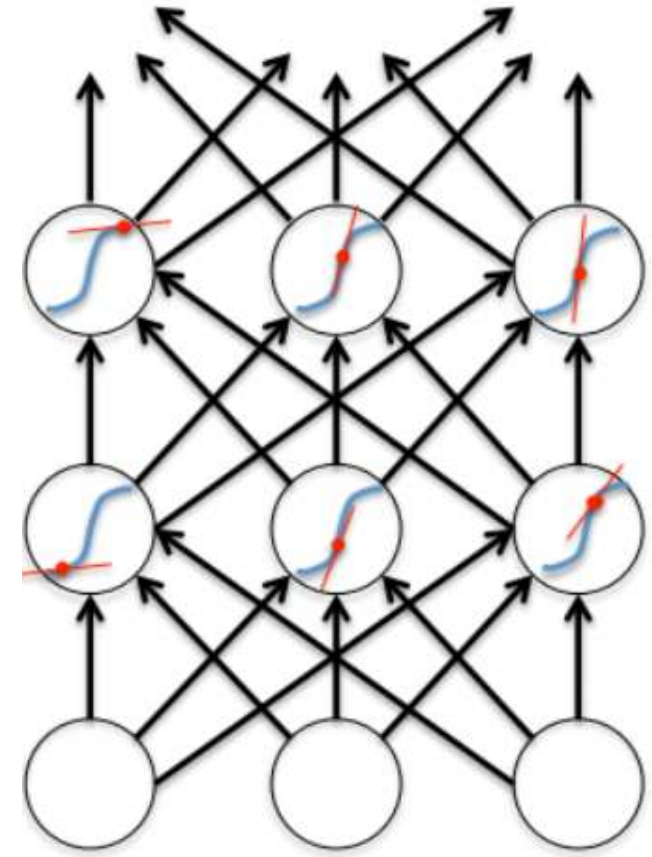
- ❑ На каждом слое вычисляется линейная комбинация входного сигнала слоя и весов каждого нейрона, в результате получается скаляр в каждом нейроне, а затем вычисляется значение функции активации.
- ❑ Функция активации, помимо необходимой нам нелинейности, помогает ограничить выходные значения нейронов. Таким образом, на выходе получается вектор, каждая компонента которого находится в заранее известных пределах.
- ❑ После того, как алгоритм вычислил градиент функции издержек относительно весов сети, он использует эти градиенты для обновления каждого веса посредством шага градиентного спуска. Таким образом, на этапе обратного прохода градиенты на выходном слое вычисляются и получают *существенные* изменения весов нейронов выходного слоя.



3.6. Проблемы исчезновения и взрывного роста градиентов.

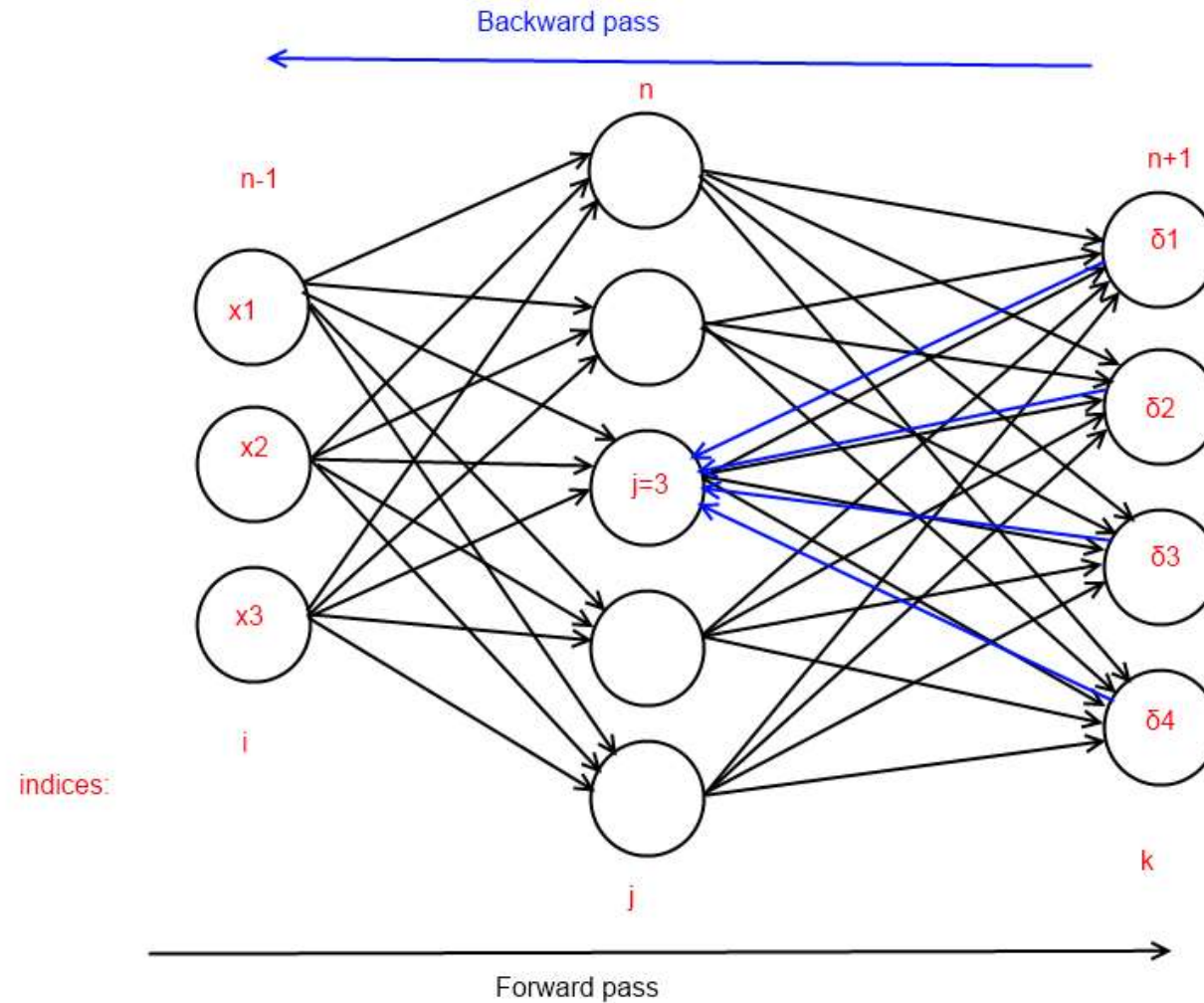
Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

- ❑ К сожалению, с продвижением алгоритма к более низким слоям градиенты зачастую становятся все меньше и меньше. В результате градиентный спуск оставляет веса связей нижних слоев практически неизменными, а обучение никогда не сходится в хорошее решение. Это называется проблемой исчезновения градиентов.
- ❑ В ряде случаев может произойти противоположное: градиенты способны расти все больше и больше, из-за чего многие слои получают безумно высокие обновления весов и алгоритм расходится. Мы имеем проблему взрывного роста градиентов, которая главным образом встречается в рекуррентных нейронных сетях.
- ❑ В целом глубокие нейронные сети страдают от изменчивых градиентов; разные слои могут обучаться с широко варьирующимися скоростями.



3.6. Проблемы исчезновения и взрывного роста градиентов. Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

□ Обратный проход для одного нейрона скрытого слоя выглядит следующим образом:



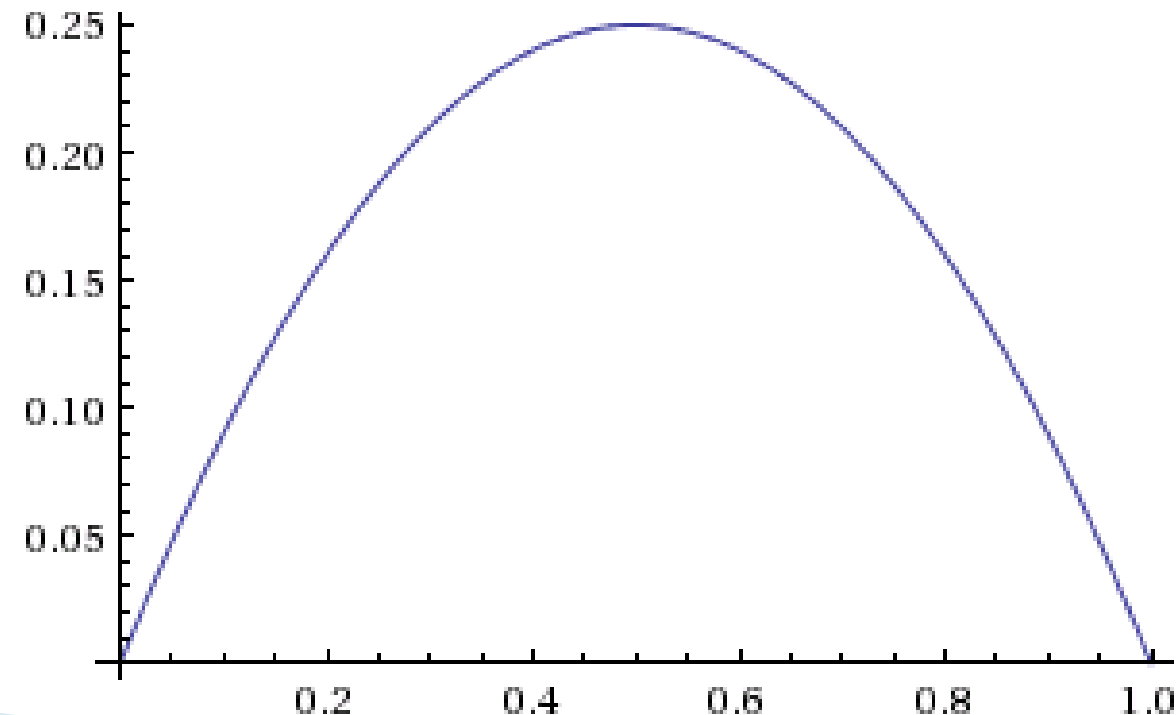
3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

- Имеем в итоге:
 - взрывной рост может случиться если веса слишком большие, или значение производной в точке слишком велико;
 - затухание происходит если значение весов или производной в точке очень мало.
- Как правило, *неглубокие сети* не страдают от этого из-за малого количества слоев, так как эффект не успевает аккумулироваться при обратном проходе.
- Если же в качестве функции активации используется сигмоид с производной следующего вида:

$$y' = f'(x) = \frac{\alpha e^{-\alpha x}}{(1 + e^{-\alpha x})^2} = \alpha f(x) (1 - f(x))$$

то при $\alpha = 1$ мы легко можем убедиться, что максимальное значение, которого достигает производная, равно 0.25 в точке $x = 0.5$, а минимальное — 0 в точках $x = 0$ и $x = 1$ (на интервале от 0 до 1, области значений логистической функции).

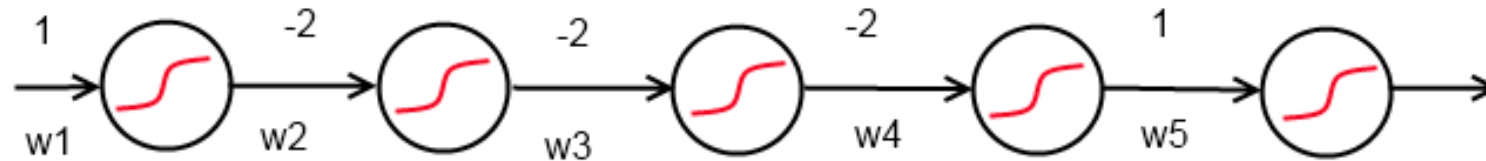


3.6. Проблемы исчезновения и взрывного роста градиентов. Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

- ❑ Следовательно, для сетей с логистической функцией активации взрывной рост и затухание в большей степени зависит от значения весов, хотя существует перманентное затухание из-за того, что в лучшем случае значение тангенса угла наклона касательной будет всего 0.25, а эти значения, от слоя к слою перемножаются:

- ❑ при очень малых значениях весов происходит затухание
- ❑ при достаточно крупных весах — рост.

Пример:



Дана простая сеть, минимизирующая квадрат Евклидова расстояния, с логистической функцией активации, найдем градиенты весов при входном образе $x = 1$ и целевом $t = 0.5$.

После прямого прохода получаем выходные значения каждого нейрона: 0.731058578630005, 0.188143725087062, 0.407022630083087, 0.307029145835665, 0.576159944190332.

Обозначим их за $y_1, \dots, y_5$.

3.6. Проблемы исчезновения и взрывного роста градиентов. Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

- $\frac{\partial E}{\partial w_5} = y_4(y_5 - t)y_5(1 - y_5) = 0.00571019949926838$
- $\frac{\partial E}{\partial w_4} = y_3 w_5 y_4 (1 - y_4)(y_5 - t)y_5(1 - y_5) = 0.0016105892898339$
- $\frac{\partial E}{\partial w_3} = y_2 w_4 y_3 (1 - y_3) w_5 y_4 (1 - y_4)(y_5 - t)y_5(1 - y_5) = -0.00035937069569135$
- $\frac{\partial E}{\partial w_2} = y_1 w_3 y_2 (1 - y_2) w_4 y_3 (1 - y_3) w_5 y_4 (1 - y_4)(y_5 - t)y_5(1 - y_5) = 0.000426583433503455$
- $\frac{\partial E}{\partial w_1} = x w_2 y_1 (1 - y_1) w_3 y_2 (1 - y_2) w_4 y_3 (1 - y_3) w_5 y_4 (1 - y_4)(y_5 - t)y_5(1 - y_5) = -0.000229451909878624$

□ Как видим, происходит затухание градиента. По ходу вычисления заметно, что аккумуляция происходит за счет перемножения весов и значений производных текущего слоя и всех последующих. Хотя начальные значения далеко не маленькие, по сравнению с тем как веса инициализируются на практике, но и этих значений недостаточно, чтобы подавить перманентное затухание от тангенсов угла наклона касательной.

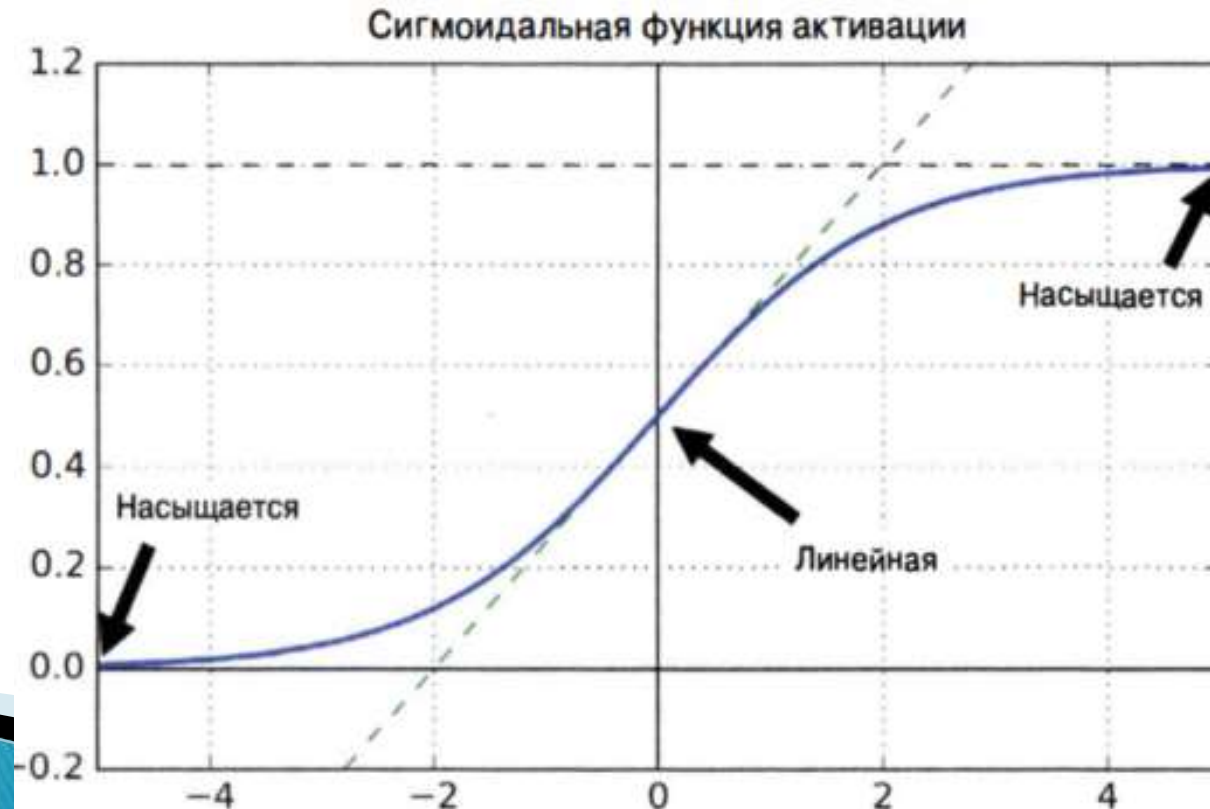
3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

- ❑ Несмотря на то что такое неподходящее поведение эмпирически наблюдалось довольно долго (и было одной из причин, по которым от нейронных сетей обычно отказывались на протяжении длительного времени), только примерно в 2010 году был совершен значительный прогресс в его понимании.
- ❑ В работе Ксавье Глоро и Йошуа Бенджио под названием "Understanding the Difficulty of Training Deep Feedforward Neural Networks" ("Осмысление сложности обучения глубоких нейронных сетей прямого распространения") обнаружился ряд подозреваемых виновников.
- ❑ В их число входило сочетание популярной логистической сигмоидальной функции активации и приема инициализации весов, который на то время был самым широко распространенным – случайной инициализации с применением нормального распределения со средним 0 и стандартным отклонением 1. Выразаясь кратко, авторы статьи показали, что с такой функцией активации и схемой инициализации дисперсия выходов каждого слоя намного больше дисперсии его входов.
- ❑ Продвигаясь вперед по сети, дисперсия продолжает увеличиваться после каждого слоя до тех пор, пока функция активации не насыщается на верхних слоях. В действительности ситуация усугубляется тем фактом, что логистическая функция имеет среднее 0.5, а не 0 (функция гиперболического тангенса имеет среднее 0 и ведет себя в глубоких сетях чуть лучше, чем логистическая функция).

3.6. Проблемы исчезновения и взрывного роста градиентов. Методы решения: инициализация весовых коэффициентов, ненасыщаемые функции активации, пакетная нормализация.

- Взглянув на логистическую функцию активации, вы можете заметить, что когда входы становятся большими (отрицательными или положительными), функция насыщается в точке 0 или 1 с производной, предельно близкой к 0. Таким образом, когда происходит обратное распространение, то у него практически нет градиента для передачи обратно через сеть, и существующий небольшой градиент продолжает понижаться с продвижением обратного распространения по верхним слоям, так что для нижних слоев фактически ничего не остается.



3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: инициализация весовых коэффициентов

Ксавье и Хе

- ❑ Глоро и Бенджио в своей статье предложили способ значительного смягчения проблемы. Нам нужно, чтобы сигнал должным образом протекал в обоих направлениях: в прямом направлении при выработывании прогнозов и в обратном направлении при обратном распространении градиентов. Мы не хотим, чтобы сигнал затухал или взрывообразно возрастал и насыщался.
- ❑ Авторы доказали, что для надлежащего протекания сигнала необходимо, чтобы дисперсия выходов каждого слоя была равна дисперсии его входов, и также нужно, чтобы градиенты имели равную дисперсию до и после протекания через слой в обратном направлении (в статье приведены математические детали).
- ❑ На самом деле гарантировать соблюдение обоих условий невозможно, если только речь не идет о слое с одинаковым количеством входных и выходных связей, но авторы представили хороший компромисс, который доказал свою эффективность на практике: веса связей должны быть инициализированы случайным образом в зависимости от $n_{\text{входов}}$ и $n_{\text{выходов}}$, как описано на следующем слайде в таблице для основных активационных функций, где $n_{\text{входов}}$ и $n_{\text{выходов}}$ – количество входных и выходных связей для слоя, чьи веса инициализируются (также называемые коэффициентом разветвления по входу (fan-in) и коэффициентом разветвления по выходу (fan-out)). Такую стратегию инициализации часто называют инициализацией Ксавье (по имени ее автора) или иногда инициализацией Глоро.

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: инициализация весовых коэффициентов

Ксавье и Хе

- ❑ Применение стратегии инициализации Ксавье может существенно ускорить обучение, и она является одним из трюков, которые привели к текущему успеху глубокого обучения.
- ❑ Стратегию инициализации для функции активации ReLU (и ее разновидностей, включая активацию ELU) называют инициализацией Хе (по фамилии ее автора).

Функция активации	Равномерное распределение $[-r, r]$	Нормальное распределение
Логистическая	$r = \sqrt{\frac{6}{n_{\text{входов}} + n_{\text{выходов}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{входов}} + n_{\text{выходов}}}}$
Гиперболического тангенса	$r = 4\sqrt{\frac{6}{n_{\text{входов}} + n_{\text{выходов}}}}$	$\sigma = 4\sqrt{\frac{2}{n_{\text{входов}} + n_{\text{выходов}}}}$
ReLU (и разновидности)	$r = \sqrt{2}\sqrt{\frac{6}{n_{\text{входов}} + n_{\text{выходов}}}}$	$\sigma = \sqrt{2}\sqrt{\frac{2}{n_{\text{входов}} + n_{\text{выходов}}}}$

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: инициализация весовых коэффициентов

Ксавье и Хе

- ❑ Примеры применения инициализации Ксавье–Глоро для нормального и равномерного распределений в полносвязном слое Keras приводятся ниже.
- ❑ Нормальное распределение:

```
tf.keras.initializers.GlorotNormal(seed=None)
```

- ❑ `seed` : целое число Python. Используется, чтобы сделать поведение инициализатора детерминированным. Обратите внимание, что инициализатор с начальным значением не будет создавать одни и те же случайные значения при нескольких вызовах, но несколько инициализаторов будут создавать одну и ту же последовательность при создании с одним и тем же начальным значением.

```
>>> # Usage in a Keras layer:  
>>> initializer = tf.keras.initializers.GlorotNormal()  
>>> layer = tf.keras.layers.Dense(3, kernel_initializer=initializer)
```

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: инициализация весовых коэффициентов

Ксавье и Хе

- ❑ Равномерное распределение:

```
tf.keras.initializers.GlorotUniform(seed=None)
```

- ❑ seed : целое число Python. Используется, чтобы сделать поведение инициализатора детерминированным. Обратите внимание, что инициализатор с начальным значением не будет создавать одни и те же случайные значения при нескольких вызовах, но несколько инициализаторов будут создавать одну и ту же последовательность при создании с одним и тем же начальным значением.

```
>>> # Usage in a Keras layer:  
>>> initializer = tf.keras.initializers.GlorotUniform()  
>>> layer = tf.keras.layers.Dense(3, kernel_initializer=initializer)
```

<https://keras.io/api/layers/initializers/>

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: ненасыщаемые функции активации

- ❑ Одна из идей, выдвинутых Глоро и Бенджио в своей статье 2010 года, заключалась в том, что проблемы исчезновения/взрывного роста градиентов возникали отчасти из-за неудачно выбранной функции активации.
- ❑ До тех пор было распространено мнение, что если уж мать-природа избрала приблизительно сигмоидальные функции активации для использования в биологических нейронах, то они обязаны быть превосходным вариантом.
- ❑ Но выяснилось, что в глубоких нейронных сетях гораздо лучше ведут себя другие функции активации, в частности функция активации ReLU. Это объясняется главным образом тем, что она не насыщается для положительных значений (а также довольно высокой скоростью вычисления).
- ❑ К сожалению, функция активации ReLU не идеальна. Она страдает от проблемы, известной как угасающие элементы ReLU (dying ReLU): во время обучения некоторые нейроны фактически отмирают, приводя к тому, что они перестают выдавать что-либо, отличающееся от 0. В ряде случаев вы можете обнаружить, что половина нейронов вашей сети погибла, особенно когда применяется большая скорость обучения. Если во время обучения веса нейрона обновляются так, что взвешенная сумма входов нейрона является отрицательной, тогда он начинает выдавать 0. После того как это произошло, возвращение к жизни нейрона маловероятно, поскольку градиент функции ReLU равен 0, когда его вход отрицателен.

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: ненасыщаемые функции активации

Leaky ReLU –линейный выпрямленный элемент с утечкой

- Чтобы решить описанную проблему, вы можете воспользоваться разновидностью функции ReLU наподобие **ReLU с утечкой (leaky ReLU)**. Гиперпараметр α задает размер "утечки" функции: это наклон функции для $x < 0$, и он обычно устанавливается в 0.01. Такой небольшой наклон гарантирует, что ReLU с утечкой никогда не угасает; нейроны могут впасть в длительную кому, но у них есть шанс со временем очнуться.

Функция:

$$f(x) = \begin{cases} \alpha x, & \text{если } x < 0 \\ x, & \text{если } x \geq 0 \end{cases}$$

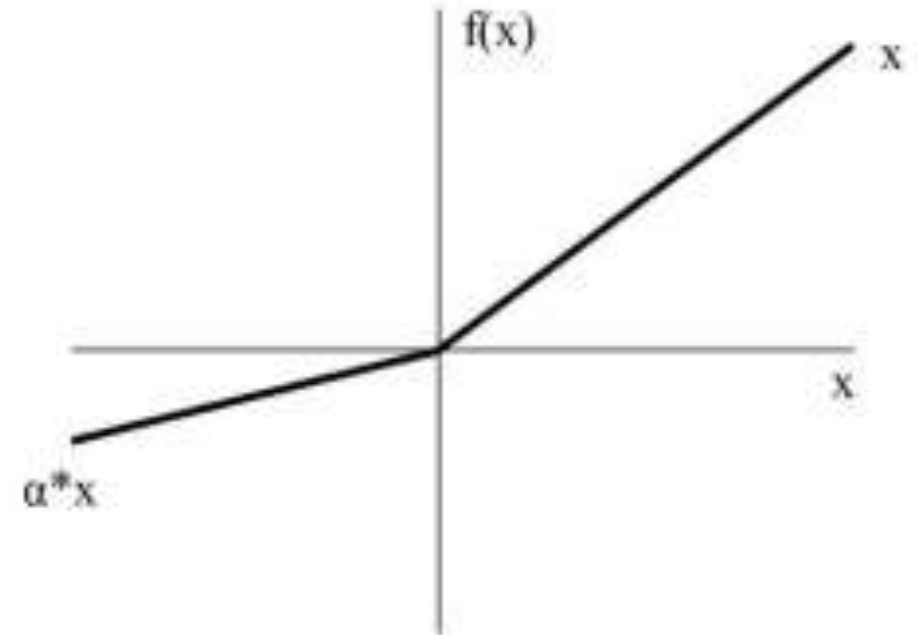
α – небольшое число (например, 0.01)

Производная:

$$f'(x) = \begin{cases} \alpha, & \text{если } x < 0 \\ 1, & \text{если } x \geq 0 \end{cases}$$

Область значений:

$(-\infty, \infty)$



3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: ненасыщаемые функции активации

Leaky ReLU –линейный выпрямленный элемент с утечкой

- ❑ На самом деле, как представляется, установка $\alpha = 0.2$ (огромная утечка) дает лучшую производительность, чем $\alpha = 0.01$ (небольшая утечка).
- ❑ Можно также применять рандомизированный вариант ReLU с утечкой (randomized leaky ReLU – RReLU), где α выбирается случайно в заданном диапазоне во время обучения и фиксируется на среднем значении во время тестирования. Он также выполняется достаточно хорошо и, кажется, действует в качестве регуляризатора (снижая риск переобучения на обучающем наборе).
- ❑ Используется также параметрический ReLU с утечкой (parametric leaky ReLU – PReLU), где α разрешено учиться во время обучения (вместо того, чтобы быть гиперпараметром, α становится параметром, который может быть модифицирован обратным распространением как любой другой параметр). Это говорило о сильном превосходстве ReLU с утечкой над ReLU на крупных наборах данных с изображениями, но на меньших наборах данных возникает риск переобучения.

3.6. Проблемы исчезновения и взрывного роста градиентов. Методы решения: ненасыщаемые функции активации

ELU – экспоненциальный линейный элемент

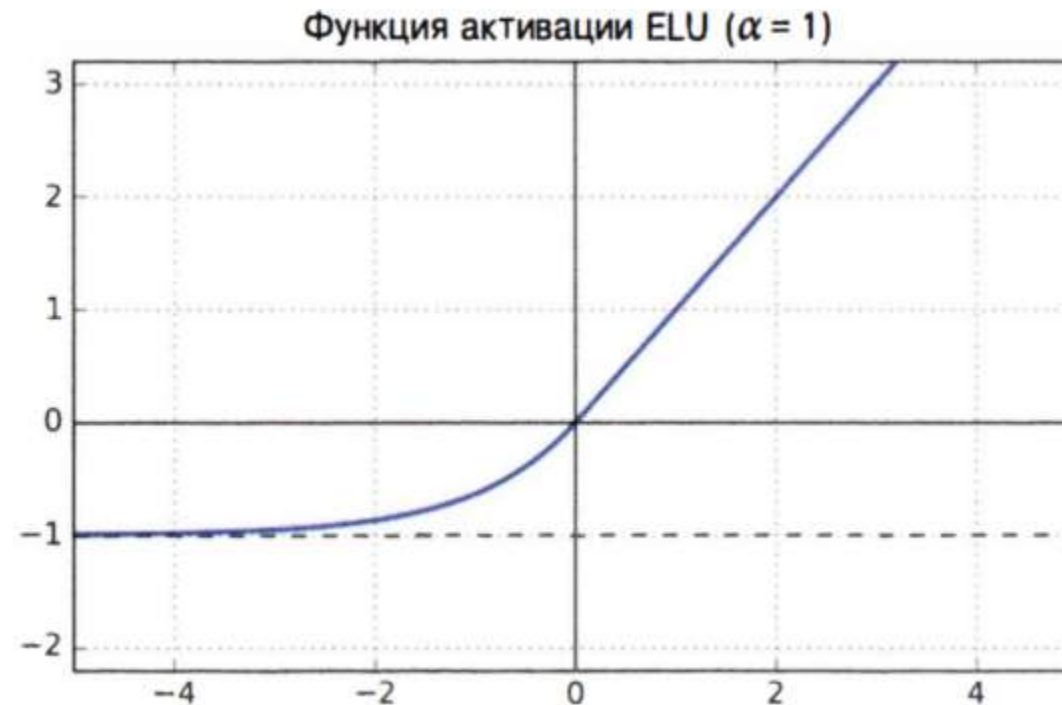
- В статье 2015 года, написанной Джорком-Арне Клевером и др., была предложена новая функция активации, названная экспоненциальным линейным элементом (exponential linear unit – ELU), которая в проведенных авторами экспериментах превзошла все разновидности ReLU: время обучения сокращалось, а нейронная сеть работала лучше на испытательном наборе.

$$f(x) = \begin{cases} \alpha(e^x - 1), & \text{если } x < 0 \\ x & , \text{если } x \geq 0 \end{cases}$$

Производная:

$$f'(x) = \begin{cases} \alpha e^x, & \text{если } x < 0 \\ 1 & , \text{если } x \geq 0 \end{cases}$$

Область значений: $(-\alpha, \infty)$



3.6. Проблемы исчезновения и взрывного роста градиентов. Методы решения: ненасыщаемые функции активации

ELU – экспоненциальный линейный элемент

- ❑ Функция активации ELU во многом похожа на функцию ReLU, но обладает несколькими важными отличиями.
- ❑ • Во-первых, она принимает отрицательные значения, когда $x < 0$, что позволяет элементу иметь средний вывод ближе к 0. Как обсуждалось ранее, это помогает смягчить проблему исчезновения градиентов. Гиперпараметр α определяет значение, к которому функция ELU приближается, когда x представляет собой большое отрицательное число. Обычно он устанавливается в 1, но при желании вы можете подстраивать его подобно любому другому гиперпараметру.
- Во-вторых, она имеет ненулевой градиент для $x < 0$, что позволяет избежать проблемы угасающих элементов.
- В-третьих, функция является повсеместно гладкой, включая точки поблизости $x = 0$, что помогает ускорить градиентный спуск, т.к. он не «отскакивает» настолько сильно слева и справа $x = 0$.

Главный недостаток функции активации ELU связан с тем, что она вычисляется медленнее, чем функция ReLU и ее разновидности (из-за применения экспоненциальной функции), но во время обучения это компенсируется более высокой скоростью схождения. Однако во время тестирования сеть ELU будет медленнее сети ReLU.

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: пакетная нормализация

- ❑ В своей статье 2015 года Сергей Иоффе и Кристиан Сегеди предложили прием, называемый пакетной нормализацией (Batch Normalization – BN), который предназначен для решения проблем исчезновения/взрывного роста градиентов и более общей проблемы, заключающейся в том, что распределение входов каждого слоя изменяется во время обучения с изменением параметров предшествующих слоев (они назвали ее проблемой внутреннего ковариационного сдвига (internal covariate shift)).
- ❑ Прием предусматривает добавление в модель операции прямо перед функцией активации каждого слоя, простое центрирование относительно нуля и нормализацию входов, а затем масштабирование и сдвиг результата с применением двух новых параметров на слой (один для масштабирования и еще один для сдвига). Другими словами, такая операция позволяет модели узнать оптимальный масштаб и среднюю величину по входам для каждого слоя.

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: пакетная нормализация

□ Чтобы центрировать относительно нуля и нормализовать входы, алгоритму необходимо произвести оценку средней величины и стандартного отклонения по входам. Он делает это путем вычисления средней величины и стандартного отклонения входов по текущему мини-пакету (отсюда и название "Пакетная нормализация"). Полная операция представлена в уравнении

$$1. \quad \mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

$$2. \quad \sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (\mathbf{x}^{(i)} - \mu_B)^2$$

$$3. \quad \hat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

$$4. \quad \mathbf{z}^{(i)} = \gamma \hat{\mathbf{x}}^{(i)} + \beta$$

- μ_B — эмпирическое среднее, вычисленное по всему мини-пакету B .
- σ_B — эмпирическое стандартное отклонение, также вычисленное по всему мини-пакету.
- m_B — количество образцов в мини-пакете.
- $\hat{\mathbf{x}}^{(i)}$ — центрированный относительно нуля и нормализованный вход.
- γ — параметр масштабирования для слоя.
- β — параметр сдвига (смещение) для слоя.
- ε — крошечное число, чтобы избежать деления на ноль (обычно 10^{-5}). Оно называется *сглаживающим членом* (*smoothing term*).
- $\mathbf{z}^{(i)}$ — выход операции BN: это масштабированная и сдвинутая версия входов.

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: пакетная нормализация

- ❑ Во время тестирования отсутствует мини-пакет, по которому можно было бы вычислить среднюю величину и стандартное отклонение, поэтому взамен просто используются средняя величина и стандартное отклонение целого обучающего набора. Обычно они эффективно вычисляются во время обучения с применением скользящего среднего. Таким образом, для каждого слоя, прошедшего пакетную нормализацию, в совокупности обучаются четыре параметра: γ (масштаб), β (смещение), μ (средняя величина) и σ (стандартное отклонение).
- ❑ Авторы продемонстрировали, что такой прием значительно улучшил все глубокие нейронные сети, с которыми они экспериментировали. Проблема исчезновения градиентов настолько уменьшилась, что они смогли использовать насыщаемые функции активации, такие как функция гиперболического тангенса или даже логистическая функция активации. Сети также стали гораздо менее чувствительными к инициализации весов.

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: пакетная нормализация

- ❑ Ниже приводится пример использования пакетной нормализации в Keras. Также используются два типа регуляризации:

```
model = keras.models.Sequential()
model.add(keras.layers.Dense(16, activation='relu', kernel_regularizer='l2', bias_regularizer='l2'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(16, activation='relu', kernel_regularizer='l2', bias_regularizer='l2'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(16, activation='relu', kernel_regularizer='l2', bias_regularizer='l2'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(16, activation='relu', kernel_regularizer='l2', bias_regularizer='l2'))
model.add(keras.layers.BatchNormalization())
model.add(layers.Dense(1, activation='sigmoid'))
```

3.6. Проблемы исчезновения и взрывного роста градиентов.

Методы решения: пакетная нормализация

□ В tensorflow пакетная нормализация может быть реализована следующим образом:

```
import tensorflow as tf

n_inputs = 28 * 28
n_hidden1 = 300
n_hidden2 = 100
n_outputs = 10

X = tf.placeholder(tf.float32, shape=(None, n_inputs), name="X")

training = tf.placeholder_with_default(False, shape=(), name='training')

hidden1 = tf.layers.dense(X, n_hidden1, name="hidden1")
bn1 = tf.layers.batch_normalization(hidden1, training=training, momentum=0.9)
bn1_act = tf.nn.elu(bn1)

hidden2 = tf.layers.dense(bn1_act, n_hidden2, name="hidden2")
bn2 = tf.layers.batch_normalization(hidden2, training=training, momentum=0.9)
bn2_act = tf.nn.elu(bn2)

logits_before_bn = tf.layers.dense(bn2_act, n_outputs, name="outputs")
logits = tf.layers.batch_normalization(logits_before_bn, training=training,
                                       momentum=0.9)
```

3.7. Современные методы и алгоритмы обучения нейронных сетей: моментная оптимизация

Вообразите большой мяч, катящийся по гладкой поверхности с пологим уклоном: он начинает движение медленно, но быстро накопит кинетическую энергию, пока в итоге не достигнет конечной скорости (если есть некоторое трение или сопротивление воздуха). Такая совершенно простая идея лежит в основе *моментной оптимизации*, предложенной Борисом Поляком в 1964 году¹⁰. По контрасту обыкновенный градиентный спуск будет делать небольшие постоянные шаги вдоль уклона, поэтому для достижения низа ему потребуется гораздо больше времени.

Вспомните, что градиентный спуск просто обновляет веса θ путем прямого вычитания градиента функции издержек $J(\theta)$ относительно весов ($\nabla_{\theta} J(\theta)$), умноженного на скорость обучения η . Вот уравнение: $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$. Он не заботится о том, какими были начальные градиенты. Если локальный градиент очень маленький, то процесс двигается крайне медленно.

3.7. Современные методы и алгоритмы обучения нейронных сетей: моментная оптимизация

Моментная оптимизация принимает во внимание предыдущие градиенты: на каждой итерации локальные градиенты вычитаются из *вектора момента* (*momentum vector*) $\mathbf{m}$, умноженного на скорость обучения η , а веса обновляются добавлением этого вектора момента (уравнение 11.4). Другими словами, градиенты используются как ускорение, а не как скорость. Для эмуляции механизма трения какого-то вида и предотвращения слишком сильного роста момента алгоритм вводит новый гиперпараметр β , называемый *моментом* (*momentum*), который должен быть установлен между 0 (высокое трение) и 1 (отсутствие трения). Типичное значение момента составляет 0.9.

Уравнение 11.4. Алгоритм моментной оптимизации

1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta + \mathbf{m}$

3.7. Современные методы и алгоритмы обучения нейронных сетей: моментная оптимизация

Вы можете легко удостовериться в том, что если градиент остается постоянным, то конечная скорость (т.е. максимальный размер обновлений весов) равна произведению данного градиента и скорости обучения η , умноженной на $\frac{1}{1-\beta}$ (с игнорированием знака). Например, если $\beta = 0.9$, тогда конечная скорость получается умножением 10 на градиент и на скорость обучения, а потому моментная оптимизация заканчивается в 10 раз быстрее градиентного спуска! Это позволяет моментной оптимизации покидать плато намного скорее, чем градиентный спуск. В частности, в главе 4 было показано, что когда входы имеют сильно отличающиеся масштабы, функция издержек будет выглядеть как вытянутая чаша (см. рис. 4.7). Градиентный спуск довольно быстро продвигается вниз по крутому уклону, но затем тратит очень много времени на движение к низу впадины. В противоположность этому моментная оптимизация будет скатываться во впадину все быстрее и быстрее, пока не достигнет низа (оптимума). В глубоких нейронных сетях, которые не применяют пакетную оптимизацию, нижние слои часто будут иметь входы с очень разными масштабами, так что использование моментной оптимизации оказывает существенную помощь. Она также может содействовать в проскакивании мимо локальных оптимумов.

3.7. Современные методы и алгоритмы обучения нейронных сетей: моментная оптимизация

Реализовать моментную оптимизацию TensorFlow очень просто: просто поменяйте `GradientDescentOptimizer` на `MomentumOptimizer` и наслаждайтесь полученными выгодами!

```
optimizer = tf.train.MomentumOptimizer(learning_rate=learning_rate,  
                                       momentum=0.9)
```

Недостаток моментной оптимизации связан с тем, что она добавляет еще один гиперпараметр, подлежащий настройке. Тем не менее, на практике значение момента 0.9 обычно работает хорошо и почти всегда обеспечивает большую скорость, чем градиентный спуск.

- ❑ В Keras метод обучения стохастического градиента с использованием моментной оптимизации, используемый при компиляции и в дальнейшем для обучения модели определяется просто:

```
16 opt = keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)  
17 model.compile(loss='categorical_crossentropy', optimizer=opt)
```

3.7. Современные методы и алгоритмы обучения нейронных сетей: ускоренный градиент Нестерова

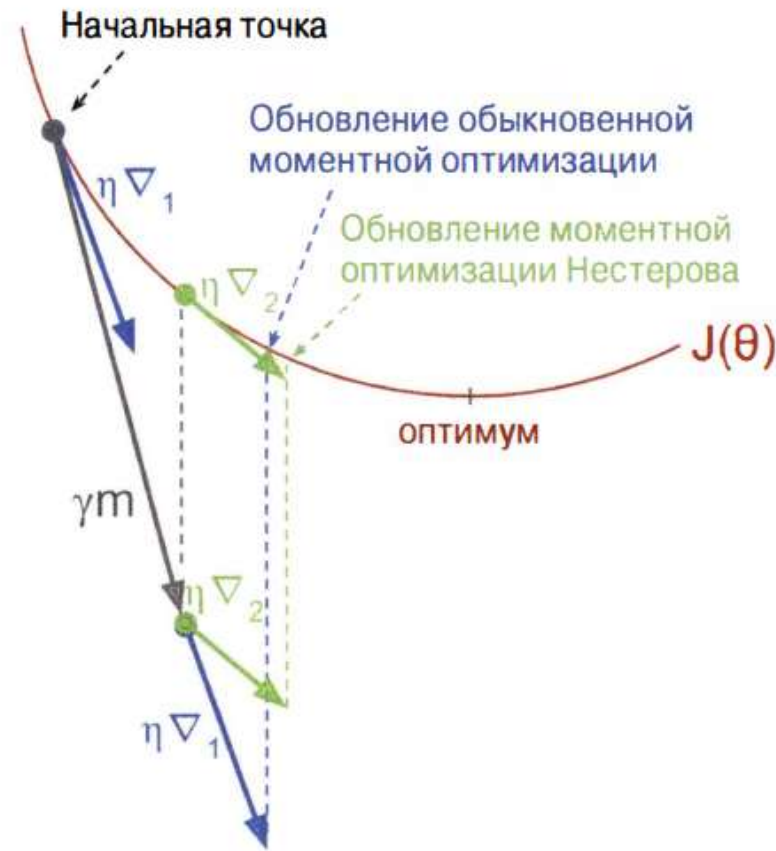
Небольшая вариация моментной оптимизации, предложенная Юрием Нестеровым в 1983 году¹¹, почти всегда быстрее обыкновенной моментной оптимизации. Идея *моментной оптимизации Нестерова*, или *ускоренного градиента Нестерова* (*Nesterov Accelerated Gradient* — *NAG*), заключается в том, чтобы измерить градиент функции издержек не в локальной позиции, но немного впереди в направлении момента (уравнение 11.5). Единственное отличие от обыкновенной моментной оптимизации в том, что градиент измеряется в точке $\theta + \beta \mathbf{m}$, а не θ .

Уравнение 11.5. Алгоритм ускоренного градиента Нестерова

1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$
2. $\theta \leftarrow \theta + \mathbf{m}$

3.7. Современные методы и алгоритмы обучения нейронных сетей: ускоренный градиент Нестерова

Такая небольшая подстройка работает из-за того, что в общем случае вектор момента будет указывать в правильном направлении (т.е. к оптимуму). Таким образом, применять градиент, измеренный чуть дальше в данном направлении, будет несколько точнее, чем использовать градиент в исходной позиции, как видно на рис. 11.6 (где ∇_1 представляет градиент функции издержек, измеренный в начальной точке θ , а ∇_2 — градиент в точке, расположенной в $\theta + \beta \mathbf{m}$).



3.7. Современные методы и алгоритмы обучения нейронных сетей: ускоренный градиент Нестерова

Легко заметить, что обновление моментной оптимизации Нестерова оказывается чуть ближе к оптимуму. Через некоторое время эти мелкие улучшения накапливаются и алгоритм NAG становится значительно быстрее обыкновенной моментной оптимизации. Кроме того, обратите внимание, что когда момент продвигает веса через впадину, ∇_1 продолжает двигаться дальше через впадину, тогда как ∇_2 двигается в направлении нижней точки впадины. Это помогает сократить колебания и в результате сделать сходжение более быстрым.

Алгоритм NAG почти всегда будет ускорять обучение в сравнении с обыкновенной моментной оптимизацией. Чтобы применить его, при создании `MomentumOptimizer` просто установите `use_nesterov=True`:

```
optimizer = tf.train.MomentumOptimizer(learning_rate=learning_rate,  
                                         momentum=0.9, use_nesterov=True)
```

- ❑ В Keras метод обучения стохастического градиента с использованием моментной оптимизации и ускоренного градиента Нестерова определяется так:

```
17 opt = keras.optimizers.SGD(  
18     learning_rate=0.1, momentum=0.9, nesterov=True, name="SGD")
```

3.7. Современные методы и алгоритмы обучения нейронных сетей: алгоритм AdaGrad

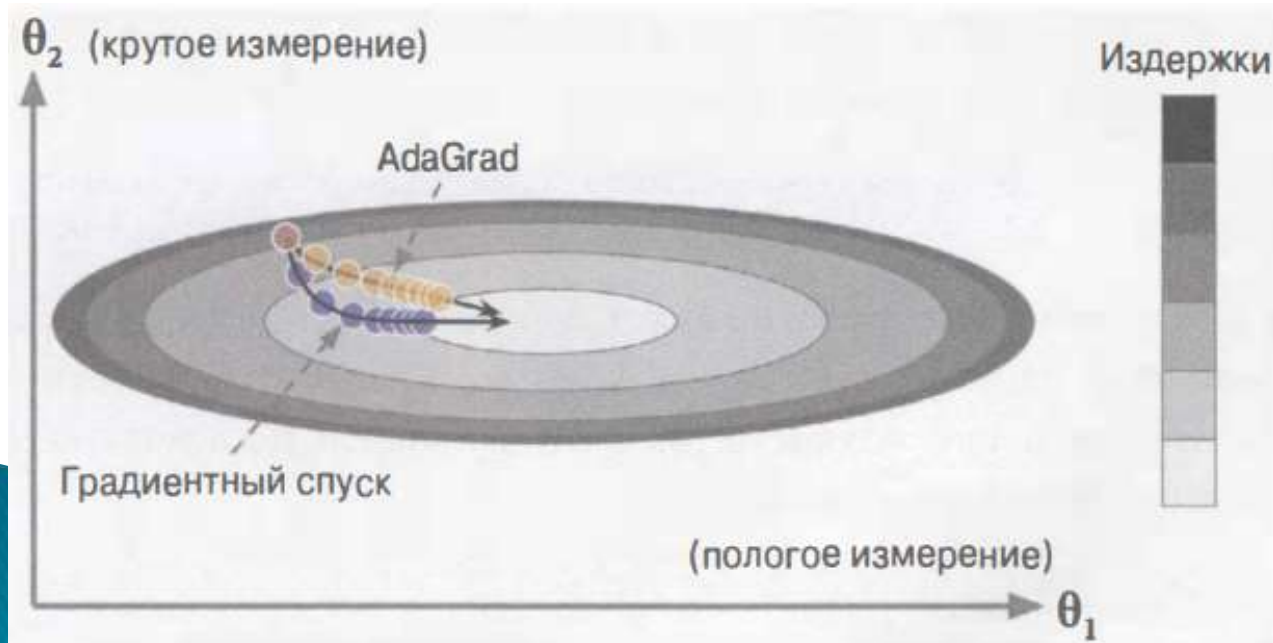
Снова рассмотрим проблему вытянутой чаши: градиентный спуск начинается с быстрого движения вниз по самому крутому уклону и затем медленно перемещается в нижнюю точку впадины. Было бы неплохо, если бы алгоритм мог своевременно выявить это и скорректировать свое направление к точке, более близкой к глобальному оптимуму.

Алгоритм *AdaGrad*¹² достигает такой цели, пропорционально уменьшая вектор-градиент вдоль самых крутых направлений (уравнение 11.6).

Уравнение 11.6. Алгоритм AdaGrad

1. $\mathbf{s} \leftarrow \mathbf{s} + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \epsilon}$

Первый шаг накапливает квадраты градиентов в векторе $\mathbf{s}$ (символ $\otimes$ представляет поэлементное умножение). Такая векторизованная форма эквивалентна вычислению $s_i \leftarrow s_i + (\partial J(\theta) / \partial \theta_i)^2$ для каждого элемента s_i вектора $\mathbf{s}$; другими словами, каждый s_i накапливает квадраты частной производной функции издержек относительно параметра θ_i . Если функция издержек является крутой вдоль i -того измерения, тогда s_i будет становиться все больше и больше на каждой итерации.



3.7. Современные методы и алгоритмы обучения нейронных сетей: алгоритм AdaGrad

Второй шаг почти идентичен градиентному спуску, но с одним большим отличием: вектор-градиент пропорционально уменьшается на коэффициент $\sqrt{s + \varepsilon}$ (символ $\oslash$ представляет поэлементное деление, а ε — сглаживающий член, позволяющий избежать деления на ноль и обычно устанавливаемый в 10^{-10}). Такая векторизованная форма эквивалентна вычислению $\theta_i \leftarrow \theta_i - \eta \partial J(\theta) / \partial \theta_i / \sqrt{s_i + \varepsilon}$ для всех параметров θ_i (одновременно).

Выражаясь кратко, алгоритм ослабляет скорость обучения, но делает это быстрее для крутых измерений, чем для измерений с более пологими уклонами. Получается то, что носит название *адаптивная скорость обучения* (*adaptive learning rate*). Она помогает направлять результирующие обновления в большей степени к глобальному оптимуму (рис. 11.7). Дополнительное преимущество связано с тем, что требуется гораздо меньший объем подстройки гиперпараметра скорости обучения η .

3.7. Современные методы и алгоритмы обучения нейронных сетей: алгоритм AdaGrad

Алгоритм AdaGrad часто работает лучше для простых квадратичных задач, но, к сожалению, при обучении нейронных сетей нередко останавливается слишком рано. Скорость обучения сокращается настолько, что алгоритм полностью прекращает работу до достижения глобального оптимума. Таким образом, хотя в TensorFlow есть класс `AdagradOptimizer`, вы не должны его использовать для обучения глубоких нейронных сетей (однако он может оказаться эффективным для более простых задач вроде линейной регрессии).

3.7. Современные методы и алгоритмы обучения нейронных сетей: алгоритм RMSProp

Несмотря на то что алгоритм AdaGrad замедляется слишком быстро и в итоге никогда не сходится в глобальном оптимуме, алгоритм *RMSProp*¹³ исправляет ситуацию, накапливая только градиенты из самых последних итераций (в противоположность накоплению всех градиентов с начала обучения). Он делает это путем применения экспоненциального ослабления на первом шаге (уравнение 11.7).

Уравнение 11.7. Алгоритм RMSProp

1. $\mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \varepsilon}$

3.7. Современные методы и алгоритмы обучения нейронных сетей: алгоритм RMSProp

Коэффициент ослабления β обычно устанавливается в 0.9. Да, снова имеем новый гиперпараметр, но такое стандартное значение часто хорошо работает, так что потребность в его подстройке может вообще не возникать. Как вы и могли ожидать, в TensorFlow имеется класс `RMSPropOptimizer`:

```
optimizer = tf.train.RMSPropOptimizer(learning_rate=learning_rate,  
                                       momentum=0.9, decay=0.9, epsilon=1e-10)
```

За исключением очень простых задач этот оптимизатор практически всегда выполняется намного лучше, чем `AdaGrad`. В действительности он был предпочтительным алгоритмом оптимизации у многих исследователей, пока не появилась оптимизация `Adam`.

❑ В Keras метод обучения RMSProp определяется так:

```
19 opt = keras.optimizers.RMSprop(  
20     learning_rate=0.001,  
21     rho=0.9,  
22     momentum=0.9,  
23     epsilon=1e-07)  
24 model.compile(loss='categorical_crossentropy', optimizer=opt)
```

или так:

```
31 model.compile(  
32     optimizer='RMSprop|',  
33     loss='binary_crossentropy',  
34     metrics=['accuracy']  
35 )
```

3.7. Современные методы и алгоритмы обучения нейронных сетей: алгоритм Adam

Оптимизация *Adam* (*adaptive moment estimation* — адаптивная оценка момента)¹⁴ объединяет идеи моментной оптимизации и RMSProp: как и моментная оптимизация, она отслеживает экспоненциально ослабляемое среднее арифметическое прошедших градиентов, и подобно RMSProp она отслеживает экспоненциально ослабляемое среднее арифметическое квадратов прошедших градиентов (уравнение 11.8)¹⁵.

Уравнение 11.8. Алгоритм Adam

1. $\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla_{\theta} J(\theta)$
2. $\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
3. $\hat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^t}$
4. $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t}$
5. $\theta \leftarrow \theta - \eta \hat{\mathbf{m}} \oslash \sqrt{\hat{\mathbf{s}} + \epsilon}$

t представляет номер итерации (начиная с 1).

Взглянув на шаги 1, 2 и 5, вы заметите близкое сходство оптимизации Adam с моментной оптимизацией и RMSProp. Единственное отличие в том, что на шаге 1 вычисляется экспоненциально ослабляемое среднее арифметическое, а не экспоненциально ослабляемая сумма, но фактически они эквивалентны за исключением постоянного множителя (ослабляемое среднее арифметическое — это просто $1 - \beta_1$ раз ослабляемой суммы). Шаги 3 и 4 представляют собой в некотором роде техническую деталь: поскольку $\mathbf{m}$ и $\mathbf{s}$ инициализируются в 0, они будут смещены в сторону 0 в начале обучения, так что указанные два шага помогут поднять $\mathbf{m}$ и $\mathbf{s}$ в начале обучения.

3.7. Современные методы и алгоритмы обучения нейронных сетей: алгоритм Adam

Гиперпараметр ослабления момента β_1 , как правило, инициализируется значением 0.9, а гиперпараметр ослабления масштабирования β_2 часто инициализируется значением 0.999. Как и ранее, сглаживающий член ε обычно инициализируется крошечным числом, скажем, 10^{-8} . Таковы стандартные значения для класса `AdamOptimizer` из TensorFlow, поэтому вы можете поступить следующим образом:

```
optimizer = tf.train.AdamOptimizer(learning_rate=learning_rate)
```

На самом деле, поскольку Adam является алгоритмом с адаптивной скоростью обучения (подобно AdaGrad и RMSProp), он требует меньшей подстройки гиперпараметра скорости обучения η . Часто можно использовать стандартное значение $\eta = 0.001$, делая применение алгоритма Adam даже более легким, чем градиентного спуска.

- В Keras метод обучения Adam определяется так:

```
24 opt = keras.optimizers.Adam(  
25     learning_rate=0.001,  
26     beta_1=0.9,  
27     beta_2=0.999,  
28     epsilon=1e-07)  
29 model.compile(loss='categorical_crossentropy', optimizer=opt)
```

или так:

```
31 model.compile(  
32     optimizer='adam',  
33     loss='binary_crossentropy',  
34     metrics=['accuracy']  
35 )
```

3.8. Регуляризация нейросетевых моделей: Компромисс между смещением и разбросом–дисперсией

- ❑ Важным теоретическим результатом статистики и машинного обучения является тот факт, что ошибка обобщения модели может быть выражена в виде суммы трех очень разных ошибок.
- ❑ **Смещение.** Эта часть ошибки обобщения связана с неверными предположениями, такими как допущение того, что данные линейные, когда они на самом деле квадратичные. Модель с высоким смещением, скорее всего, не дообучится на обучаемых данных.
- ❑ **Дисперсия – разброс.** Эта часть объясняется чрезмерной чувствительностью модели к небольшим изменениям в обучающих данных. Модель со многими степенями свободы (такая как нейронная сеть), вероятно, будет иметь высокую дисперсию и потому способна переобучаться на обучающих данных.
- ❑ **Неустраняемая погрешность.** Эта часть появляется вследствие зашумленности самих данных. Единственный способ сократить неустраняемую погрешность в ошибке предусматривает очистку данных (приведение в порядок источников данных, таких как неисправные датчики, или выявление и устранение выбросов).

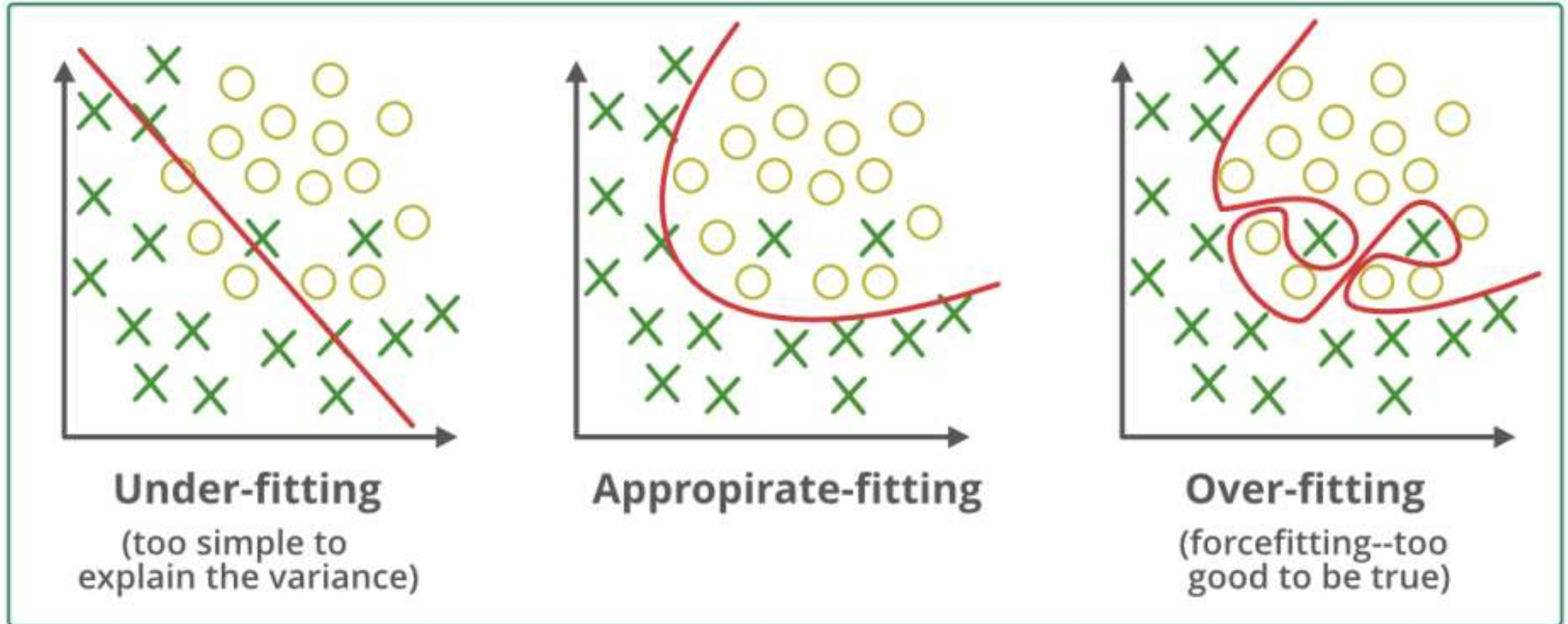
3.8. Регуляризация нейросетевых моделей: Компромисс между смещением и разбросом-дисперсией

Задача:



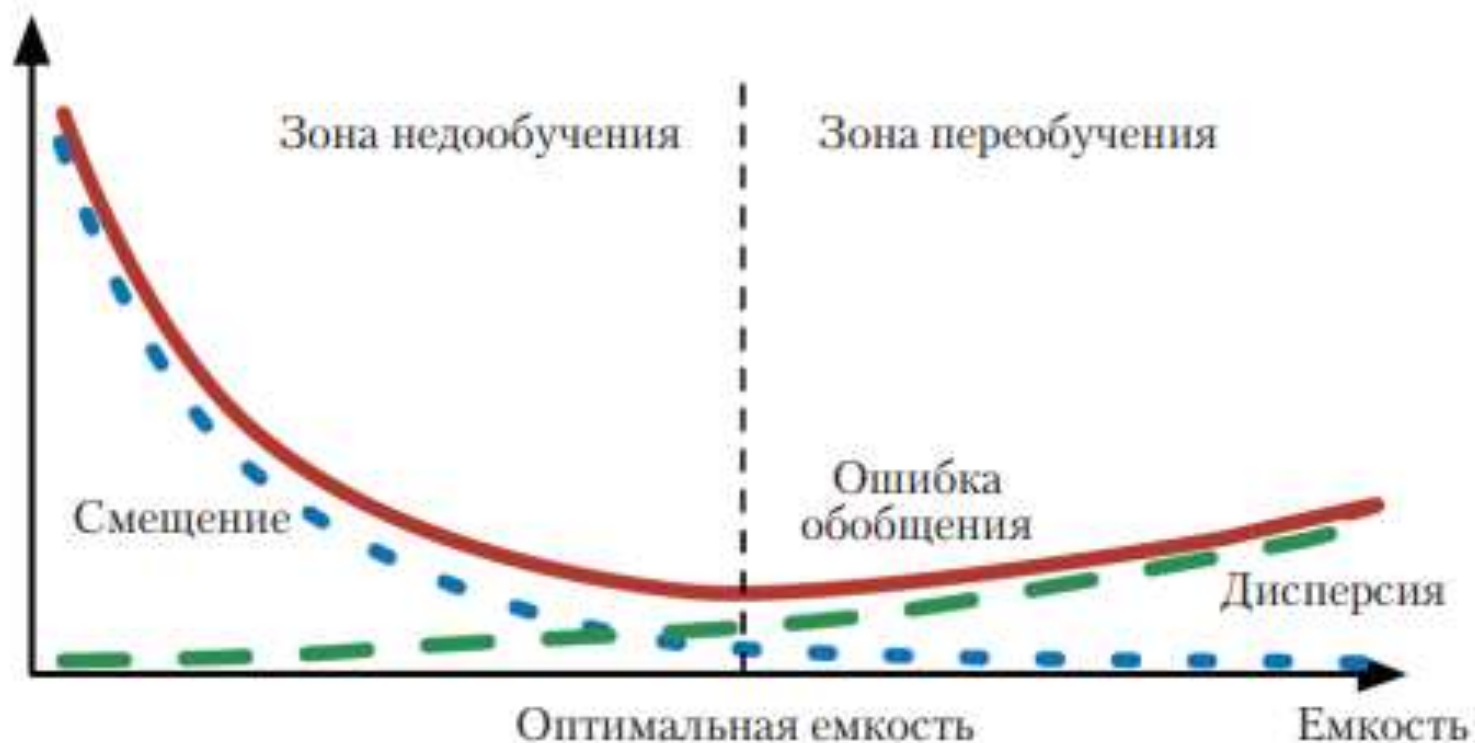
Ошибка на обучающей выборке	1%	15%	15%	0.5%
Ошибка на валидационной выборке	11%	16%	30%	1%
Результат	Высокий разброс	Высокое смещение	Высокое смещение и разброс	Низкое смещение и разброс

3.8. Регуляризация нейросетевых моделей: Компромисс между смещением и разбросом-дисперсией



3.8. Регуляризация нейросетевых моделей: Компромисс между смещением и разбросом-дисперсией

- С ростом емкости-сложности модели (ось x) смещение (пунктирная линия) уменьшается, а дисперсия (штриховая линия) увеличивается, в результате чего получается еще одна U-образная кривая для ошибки обобщения (жирная линия). В какой-то точке на оси x емкость оптимальна, слева от этой точки имеет место недообучение, справа – переобучение. Это соотношение аналогично соотношению между емкостью-сложностью модели, недообучением и переобучением.



3.8. Регуляризация нейросетевых моделей: раннее прекращение, динамическое изменение скорости обучения и применение обратных вызовов для воздействия на модель в ходе обучения

- ❑ Глубокие нейронные сети обычно имеют десятки тысяч параметров, а иногда параметров миллионы. Благодаря настолько большому количеству параметров сеть обладает невероятной степенью свободы и может подстраиваться к широкому многообразию сложных наборов данных. Но такая огромная гибкость означает и предрасположенность к переобучению на обучающем наборе.
- ❑ Замечательное решение, позволяющее избежать переобучения, заключается в раннем прекращении (рассмотренном ранее для линейных регрессионных моделей): просто прервите обучение, когда его производительность на проверочном (валидационном) наборе начинает падать.
- ❑ Хотя раннее прекращение на практике работает очень хорошо, вы обычно можете добиться гораздо большей производительности от сети, сочетая его с другими приемами регуляризации.

3.8. Регуляризация нейросетевых моделей: раннее прекращение, динамическое изменение скорости обучения и применение обратных вызовов для воздействия на модель в ходе обучения

- ❑ Раннее прекращение для нейросетевых моделей можно реализовать с использованием механизма обратных вызовов в Keras.
- ❑ *Обратный вызов* — это объект (экземпляр класса, реализующего конкретные методы), который передается в модель через вызов `fit` и который будет вызываться моделью в разные моменты в процессе обучения. Он имеет доступ ко всей информации о состоянии модели и ее качестве и может предпринимать следующие действия: прерывать обучение, сохранять модели, загружать разные наборы весов или как-то иначе изменять состояние модели.
- ❑ Вот несколько примеров использования обратных вызовов:
 - ❑ *фиксация состояния модели в контрольных точках* — сохранение текущих весов модели в разные моменты в ходе обучения;
 - ❑ *раннее прекращение* — прерывание обучения, когда оценка потерь на проверочных данных перестает улучшаться (и, конечно, сохранение лучшего варианта модели, полученного в ходе обучения);
 - ❑ *динамическая корректировка значений некоторых параметров в процессе обучения*, например шага обучения оптимизатора.
 - ❑

3.8. Регуляризация нейросетевых моделей: раннее прекращение, динамическое изменение скорости обучения и применение обратных вызовов для воздействия на модель в ходе обучения

- ❑ Модуль `keras.callbacks` включает в себя ряд встроенных обратных вызовов. Рассмотрим основные из них, чтобы получить представление о том, как ими пользоваться:

`keras.callbacks.ModelCheckpoint`

`keras.callbacks.EarlyStopping`

`keras.callbacks.ReduceLROnPlateau`

3.8. Регуляризация нейросетевых моделей: раннее прекращение, динамическое изменение скорости обучения и применение обратных вызовов для воздействия на модель в ходе обучения

- ❑ **Обратные вызовы *ModelCheckpoint* и *EarlyStopping***
- ❑ Обратный вызов *EarlyStopping* можно использовать для прерывания процесса обучения, если находящаяся под наблюдением целевая метрика не улучшалась на протяжении заданного количества эпох. Например, этот обратный вызов позволит прервать обучение после наступления эффекта переобучения и тем самым избежать повторного обучения модели в течение меньшего количества эпох. Этот обратный вызов обычно используется в комбинации с *ModelCheckpoint*, который позволяет сохранять состояние модели в ходе обучения (и, при необходимости, сохранять только лучшую модель: версию модели, достигшую лучшего качества к концу эпохи):

3.8. Регуляризация нейросетевых моделей: раннее прекращение, динамическое изменение скорости обучения и применение обратных вызовов для воздействия на модель в ходе обучения

Передача обратных вызовов в модель через аргумент `callbacks` метода `fit` в виде списка. Вы можете передать любое количество обратных вызовов

```
import keras
```

```
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor='val_acc',  
        patience=1,  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath='my_model.h5',  
        monitor='val_loss',  
        save_best_only=True,  
    )  
]
```

Прерывание обучения, когда качество модели перестает улучшаться

Слежение за изменением точности модели на проверочных данных

Прерывание обучения, когда точность не улучшается дольше чем в течение одной эпохи (другими словами, в течение двух эпох)

Сохранение текущих весов после каждой эпохи

Путь к файлу модели

Эти аргументы требуют, чтобы файл модели не затирался, если значение `val_loss` не улучшилось, что позволяет сохранять только лучшую модель

```
model.compile(optimizer='rmsprop',  
              loss='binary_crossentropy',  
              metrics=['acc'])
```

Мы следим за точностью, поэтому она должна быть частью набора метрик модели

```
model.fit(x, y,  
          epochs=10,  
          batch_size=32,  
          callbacks=callbacks_list,  
          validation_data=(x_val, y_val))
```

Обратите внимание: поскольку обратный вызов следит за потерями и точностью на проверочных данных, мы должны передать `validation_data` в вызов `fit`

3.8. Регуляризация нейросетевых моделей: раннее прекращение, динамическое изменение скорости обучения и применение обратных вызовов для воздействия на модель в ходе обучения

❑ Обратный вызов *ReduceLROnPlateau*

- ❑ Этот обратный вызов можно использовать для снижения скорости обучения, когда потери на проверочных данных перестают уменьшаться. Уменьшение или увеличение скорости обучения в точке перегиба кривой потерь — эффективная стратегия выхода из локального минимума в ходе обучения. Следующий пример демонстрирует применение обратного вызова *ReduceLROnPlateau*:

```
callbacks_list = [  
    keras.callbacks.ReduceLROnPlateau(  
        monitor='val_loss'  ← Мониторинг потерь модели на проверочных данных  
        factor=0.1,         ← Уменьшение скорости обучения в 10 раз  
        patience=10,        ← Произведение вызова, когда потери на проверочных  
    )                        данных не улучшаются в течение 10 эпох  
]
```

```
model.fit(x, y,  
          epochs=10,  
          batch_size=32,  
          callbacks=callbacks_list,  
          validation_data=(x_val, y_val))
```

Поскольку обратный вызов следит за потерями на проверочных данных, мы должны передать `validation_data` в вызов `fit`

3.8. Регуляризация нейросетевых моделей: l1, l2 регуляризация

- Подобно тому, как мы поступали в отношении простых линейных моделей, вы можете применять регуляризацию *l1* и *l2* для ограничения весов связей нейронной сети.

Функция стоимости:

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|_F^2$$

Норма Фробениуса:

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{ij}^{[l]})^2$$

Размеры *l*-го слоя:

$$w^{[l]} : (n^{[l]}, n^{[l-1]})$$

- Для регуляризации *l1* берется норма по модулю:

$$\lambda \sum_w |w|$$

3.8. Регуляризация нейросетевых моделей: l1, l2 регуляризация

Градиентный спуск:

$$\frac{\partial J}{\partial w^{[l]}} = dw^{[l]}$$

$$dw^{[l]} = (\text{из обратного распространения ошибки}) + \frac{\lambda}{m} w^{[l]}$$

$$w^{[l]} = w^{[l]} - \alpha dw^{[l]}$$

$$w^{[l]} = w^{[l]} - \alpha [(\text{из обратного распространения ошибки}) + \frac{\lambda}{m} w^{[l]}]$$

$$w^{[l]} = w^{[l]} - \alpha (\text{из обратного распространения ошибки}) - \frac{\alpha \lambda}{m} w^{[l]}$$

$$w^{[l]} = (1 - \frac{\alpha \lambda}{m}) w^{[l]} - \alpha (\text{из обратного распространения ошибки})$$

$$(1 - \frac{\alpha \lambda}{m}) < 1$$

Такая процедура также называется “убывание весов” (weight decay)

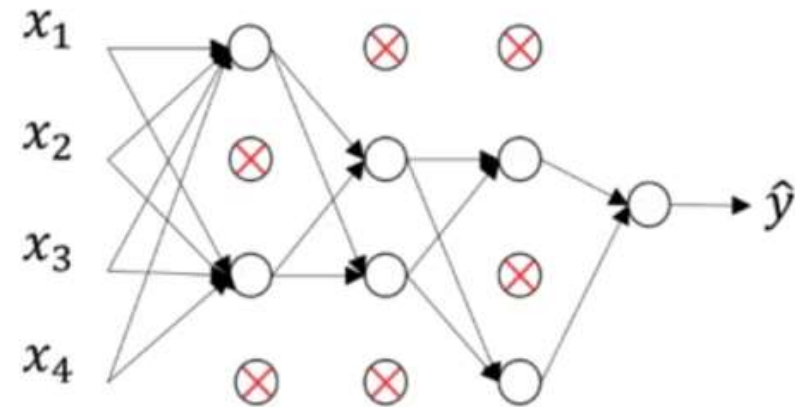
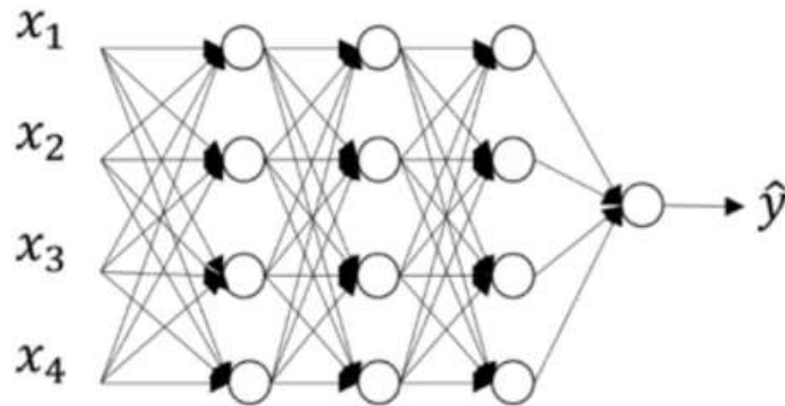
3.8. Регуляризация нейросетевых моделей: l1, l2 регуляризация

Функция стоимости:

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|_F^2$$

Некоторые из матриц весов становятся почти нулевыми:

$$w^{[l]} \approx 0$$



3.8. Регуляризация нейросетевых моделей: l1, l2 регуляризация

- В Keras регуляризация весов осуществляется путем передачи в слои именованных аргументов с *экземплярами регуляризаторов весов*:

```
from keras import regularizers

regularizers.l1(0.001)    ← L1-регуляризация

regularizers.l1_l2(l1=0.001, l2=0.001) ← Объединенная L1- и L2-регуляризация

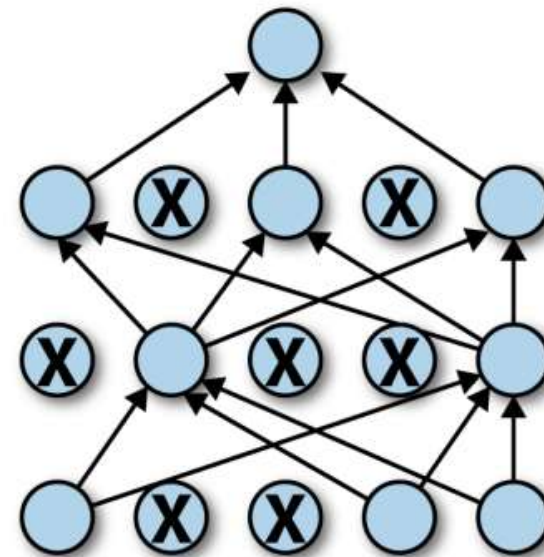
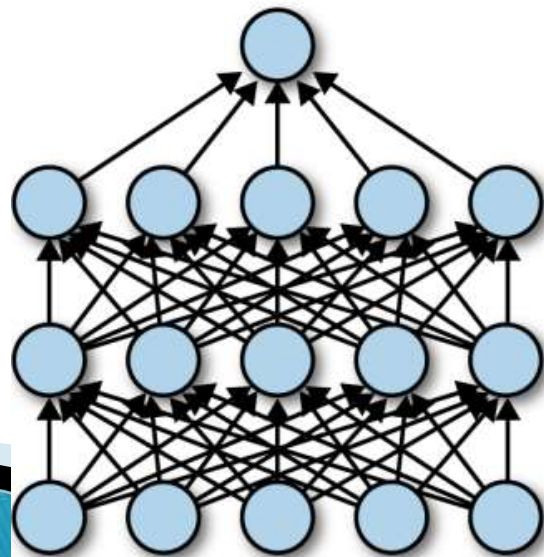
from keras import regularizers

model = models.Sequential()
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),
                        activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),
                        activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

$l2(0.001)$ означает, что каждый коэффициент в матрице весов слоя будет добавлять $0.001 * \text{weight_coefficient_value}$ в общее значение потерь сети. Обратите внимание: так как *штраф добавляется только на этапе обучения*, величина потерь сети на этапе обучения будет намного выше, чем на этапе контроля.

3.8. Регуляризация нейросетевых моделей: регуляризация исключением – dropout

- ❑ Наиболее популярным приемом регуляризации для глубоких нейронных сетей является **отключение (dropout)**.
- ❑ Оно было предложено Джефффри Хинтоном в 2012 году и дополнительно детализировано в статье Нитиша Шриваставы и др. Отключение доказало свою высокую успешность:
 - ❑ из-за его добавления даже современные нейронные сети получают рост точности в 1-2%. Рост может не выглядеть большим, но когда модуль уже имеет 95%-ную точность, то рост точности в 2% означает уменьшение частоты ошибок почти на 40% (от 5%-ной ошибки до приблизительно 3%-ной).
 - ❑ Алгоритм довольно прост: на каждом шаге обучения каждый нейрон (в том числе входные, но не выходные нейроны) имеет вероятность p быть временно "отключенным": так что он будет полностью игнорироваться в течение этого шага обучения, но оказаться активным во время следующего шага. Гиперпараметр p называется **долей отключения (dropout rate)** и обычно устанавливается в 50%. После обучения нейроны больше не отключаются.



3.8. Регуляризация нейросетевых моделей: регуляризация исключением – dropout

- ❑ Нейроны, обученные с отключением, не могут совместно адаптироваться с соседними нейронами; они должны быть максимально полезными сами по себе.
- ❑ Нейроны, обученные с отключением, также не могут чрезмерно полагаться лишь на несколько входных нейронов; они обязаны обращать внимание на каждый из своих входных нейронов. В итоге они становятся менее чувствительными к незначительным изменениям во входах.
- ❑ В конечном счете, вы получаете более надежную сеть, которая лучше обобщается.
- ❑ Еще один способ понять мощь отключения – осознать, что на каждом шаге обучения генерируется уникальная нейронная сеть. Поскольку каждый нейрон может либо присутствовать, либо отсутствовать, то существует всего 2^N возможных сетей (где N – общее количество допускающих отбрасывание нейронов). Это настолько гигантское число, что выбрать дважды одну и ту же нейронную сеть практически невозможно. После прогона 10000 шагов обучения вы, по сути, обучили 10 000 разных нейронных сетей (каждую с только одним обучающим образцом).
- ❑ Очевидно, что такие нейронные сети не являются независимыми, т.к. разделяют многие свои веса, но все-таки они все разные. Результирующую нейронную сеть можно рассматривать как усредненный ансамбль всех этих более мелких нейронных сетей.

3.8. Регуляризация нейросетевых моделей: регуляризация исключением – dropout

- ❑ Чтобы реализовать отключение, используя TensorFlow, вы можете просто применить функцию `tf.layers.dropout()` к входному слою и/или к выходу любого желаемого скрытого слоя. Во время обучения указанная функция случайным образом отключает некоторые элементы (устанавливая их в 0) и разделяет оставшиеся элементы на вероятность сохранения. После обучения эта функция вообще ничего не делает. Следующий код применяет регуляризацию на основе отключения к нашей нейронной сети с тремя слоями:
- ❑ Конечно, как это делалось при пакетной нормализации, во время обучения `training` необходимо установить в `True`, а во время испытаний оставить стандартное значение `False`.
- ❑ Если вы обнаруживаете, что модель переобучается, тогда можете увеличить долю отключения. И наоборот, если модель не дообучается на обучающем наборе, то долю отключения потребуется уменьшить. Также может помочь увеличение доли отключения для крупных слоев и ее уменьшение для небольших слоев.
- ❑ Отключение имеет тенденцию значительно замедлять сходжение, но при надлежащей подстройке оно обычно приводит к гораздо лучшей модели.

```
training=tf.placeholder_with_default(False,shape=(),name='training')
dropout_rate = 0.5 # == 1 - keep_prob
X_drop = tf.layers.dropout(X, dropout_rate, training=training)
with tf.name_scope("dnn"):
    hidden1 = tf.layers.dense(X_drop, n_hidden1,
                              activation=tf.nn.relu, name="hidden1")
    hidden1_drop = tf.layers.dropout(hidden1, dropout_rate,
                                     training=training)
    hidden2 = tf.layers.dense(hidden1_drop, n_hidden2,
                              activation=tf.nn.relu, name="hidden2")
    hidden2_drop = tf.layers.dropout(hidden2, dropout_rate,
                                     training=training)
    logits = tf.layers.dense(hidden2_drop, n_outputs,
                              name="outputs")
```

3.8. Регуляризация нейросетевых моделей: регуляризация исключением – dropout

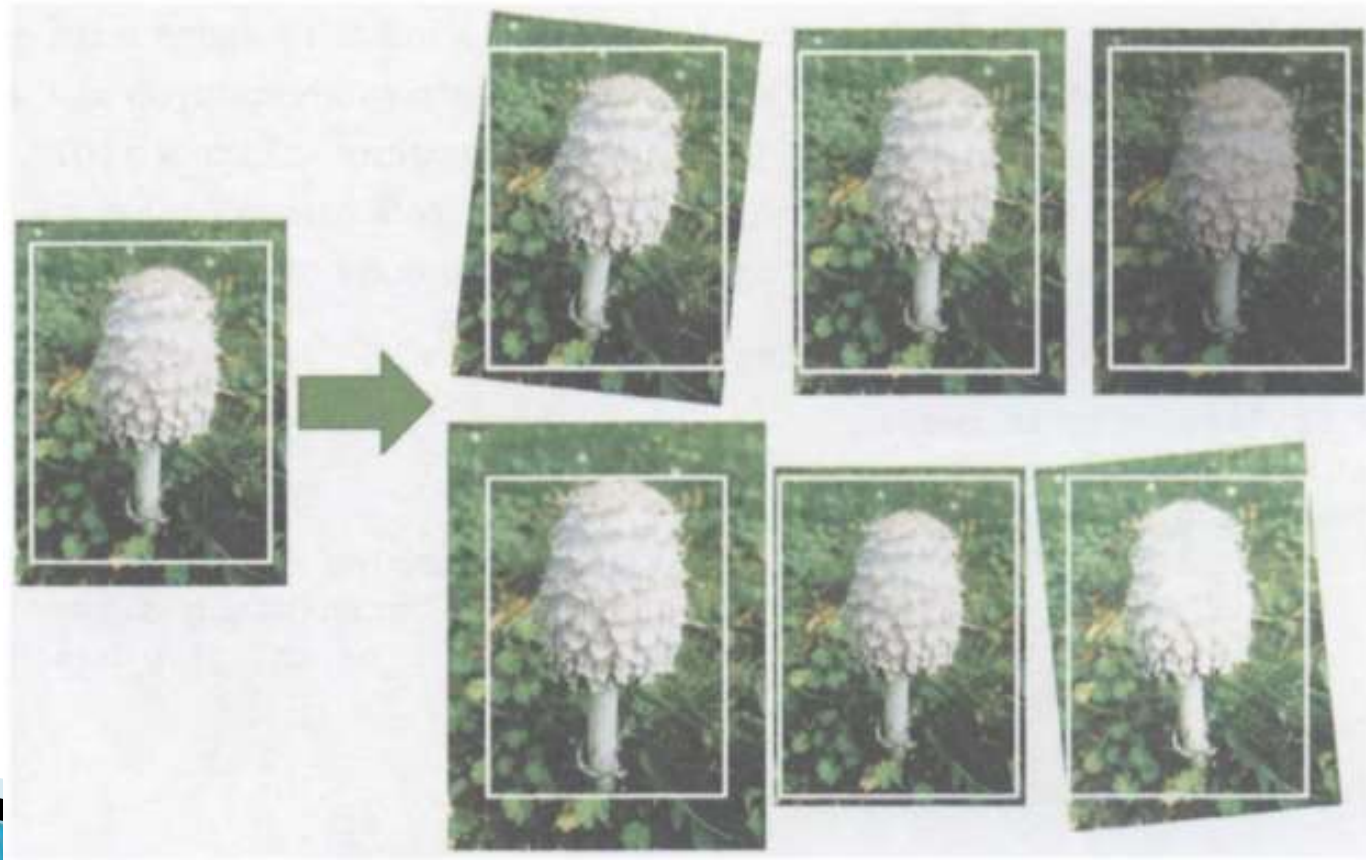
```
model = tf.keras.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(400, 200, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dropout(0.25))
model.add(layers.Dense(512, kernel_regularizer=regularizers.l1_l2(l1=0.00001, l2=0.001, ), activation='relu'))
model.add(layers.Dense(128, kernel_regularizer=regularizers.l1_l2(l1=0.00001, l2=0.001, ), activation='relu'))
model.add(layers.Dense(11, activation='softmax')) # 'sigmoid'
```

3.8. Регуляризация нейросетевых моделей: регуляризация дополнением данных

- ❑ Последний прием регуляризации – дополнение данных, предусматривает генерирование новых обучающих образцов на основе существующих, что искусственно повышает размер обучающего набора.
- ❑ Такое действие будет уменьшать переобучение и это делает его приемом регуляризации. Хитрость в том, чтобы генерировать реалистичные обучающие образцы; в идеале человек не должен быть способен сказать, какие образцы были сгенерированы, а какие нет. Кроме того, простое добавление белого шума не поможет; применяемые вами модификации обязаны быть поддающимися изучению (белый шум не поддается).
- ❑ Например, если ваша модель предназначена для классификации фотографий грибов, тогда вы можете слегка сдвинуть, повернуть и изменить размер каждой фотографии из обучающего набора на различные величины, после чего добавить результирующие фотографии в обучающий набор. Это вынудит модель стать более толерантной к позиции, ориентации и размеру грибов на фотографии. Если вы хотите, чтобы модель была более толерантной к условиям освещения, тогда можете похожим образом сгенерировать множество изображений с разными контрастами. Предполагая, что грибы симметричны, вы также можете переворачивать фотографии по горизонтали. За счет сочетания указанных трансформаций вы в состоянии значительно увеличить свой обучающий набор.

3.8. Регуляризация нейросетевых моделей: регуляризация дополнением данных

- ❑ Библиотеки Keras и TensorFlow предлагают несколько операций манипулирования изображениями, в том числе перемещение (сдвиг), поворот, изменение размера, переворачивание и обрезка, а также регулирование яркости, контрастности, насыщенности и оттенка. Они облегчают реализацию дополнения данных для наборов данных с изображениями.
- ❑ Пример применения метода дополнения данных – аугментации инструментами Keras будет рассмотрен далее при рассмотрении сверточных нейронных сетей в задаче классификации изображений.



3.9 Примеры решения задач регрессионного и классификационного анализа последовательностей данных полносвязными нейронными сетями. 3.9.1

Пример решения задачи регрессии полносвязной сетью

□ Сформируем набор данных, обучающую, валидационную и тестовую выборки:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
test_size = 0.2
val_size = 0.25
seed = 7
X = 2 - 3 * np.random.normal(0, 1, 50)
Y = X - 2 * (X ** 2) + 0.5 * (X ** 3) + np.exp(-X)+np.random.normal(-3, 3, 50)
# transforming the data to include another axis
X = X[:, np.newaxis]
Y = Y[:, np.newaxis]
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,\
test_size=test_size, random_state=seed)
X_train, X_val, Y_train, Y_val = train_test_split(X, Y,\
test_size=test_size, random_state=seed)
```

3.9.1 Пример решения задачи регрессии полносвязной сетью

□ Построим программную модель полносвязной нейронной сети:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import models
from tensorflow.keras import layers
from tensorflow.keras import optimizers

import random
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)

from keras import Input
active1 = 'elu'#'leaky_relu'
active2 = 'linear'
model = keras.models.Sequential()
model.add(layers.Dense(128, activation=active1, input_shape=(X.shape[1],)))
model.add(layers.Dense(256, activation=active1))
model.add(layers.Dense(512, activation=active1))
model.add(layers.Dense(256, activation=active1))
model.add(layers.Dense(128, activation=active1))
model.add(layers.Dense(56, activation=active1))
model.add(keras.layers.Dense(1, activation=active2))

model.summary()
```

3.9.1 Пример решения задачи регрессии полносвязной сетью

□ Рассмотрим summary модели полносвязной нейронной сети:

Model: "sequential_59"

Layer (type)	Output Shape	Param #
dense_337 (Dense)	(None, 128)	256
dense_338 (Dense)	(None, 256)	33024
dense_339 (Dense)	(None, 512)	131584
dense_340 (Dense)	(None, 256)	131328
dense_341 (Dense)	(None, 128)	32896
dense_342 (Dense)	(None, 56)	7224
dense_343 (Dense)	(None, 1)	57

Total params: 336369 (1.28 MB)
Trainable params: 336369 (1.28 MB)
Non-trainable params: 0 (0.00 Byte)

3.9.1 Пример решения задачи регрессии полносвязной сетью

❑ Откомпилируем модель полносвязной нейронной сети:

```
model.compile(optimizer='adam',  
loss='mse',  
metrics=[tf.keras.metrics.RootMeanSquaredError()])  
  
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor='val_root_mean_squared_error',  
        patience=25,  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath='Dense_Regression_17.09.2024.h5',  
        monitor='val_root_mean_squared_error',  
        save_best_only=True,  
    ),  
    keras.callbacks.ReduceLROnPlateau(  
        monitor='val_loss',  
        factor=0.25,  
        patience=10,  
    )  
]
```

3.9.1 Пример решения задачи регрессии полносвязной сетью

□ Обучим с валидацией модель полносвязной нейронной сети:

```
m=model.fit(X_train, Y_train, validation_data=(X_val, Y_val), callbacks=callbacks_list,  
            epochs=250, batch_size=16)
```

```
Epoch 48/250  
3/3 [=====] - 0s 14ms/step - loss: 5.2759 - root_mean_squared_error: 2.2805 - val_loss: 7.7775 - val  
_root_mean_squared_error: 2.7888 - lr: 2.5000e-04  
Epoch 49/250  
3/3 [=====] - 0s 16ms/step - loss: 5.4369 - root_mean_squared_error: 2.3317 - val_loss: 7.1617 - val  
_root_mean_squared_error: 2.6761 - lr: 2.5000e-04  
Epoch 50/250  
3/3 [=====] - 0s 14ms/step - loss: 5.2259 - root_mean_squared_error: 2.2860 - val_loss: 7.0346 - val  
_root_mean_squared_error: 2.6523 - lr: 6.2500e-05  
Epoch 51/250  
3/3 [=====] - 0s 15ms/step - loss: 5.1600 - root_mean_squared_error: 2.2716 - val_loss: 6.8943 - val  
_root_mean_squared_error: 2.6257 - lr: 6.2500e-05  
Epoch 52/250  
3/3 [=====] - 0s 15ms/step - loss: 5.1409 - root_mean_squared_error: 2.2674 - val_loss: 6.8191 - val  
_root_mean_squared_error: 2.6113 - lr: 6.2500e-05  
Epoch 53/250  
3/3 [=====] - 0s 15ms/step - loss: 5.1650 - root_mean_squared_error: 2.2727 - val_loss: 6.8932 - val  
_root_mean_squared_error: 2.6255 - lr: 6.2500e-05  
Epoch 54/250  
3/3 [=====] - 0s 14ms/step - loss: 5.1300 - root_mean_squared_error: 2.2649 - val_loss: 7.0261 - val  
_root_mean_squared_error: 2.6507 - lr: 6.2500e-05
```

3.9.1 Пример решения задачи регрессии полносвязной сетью

□ Протестируем модель полносвязной нейронной сети:

```
test_loss, test_acc = model.evaluate(X_test, Y_test)
print('test_loss:', test_loss, ' ', 'test_acc:', test_acc)
```

```
Y_test_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(Y_test, Y_test_pred))
r2 = r2_score(Y_test, Y_test_pred)
```

```
print(rmse)
print(r2)
print(Y_test)
print(Y_test_pred)
```

```
1/1 [=====] - 0s 18ms/step - loss: 7.0261 - root_mean_squared_error: 2.6507
test_loss: 7.026129245758057    test_acc: 2.6506845951080322
1/1 [=====] - 0s 56ms/step
2.6506843748891513
0.9862429077465396
```

3.9.1 Пример решения задачи регрессии полносвязной сетью

□ Вывод предиктивной и эталонной выходных последовательностей:

Y_test – эталон

Y_test_pred

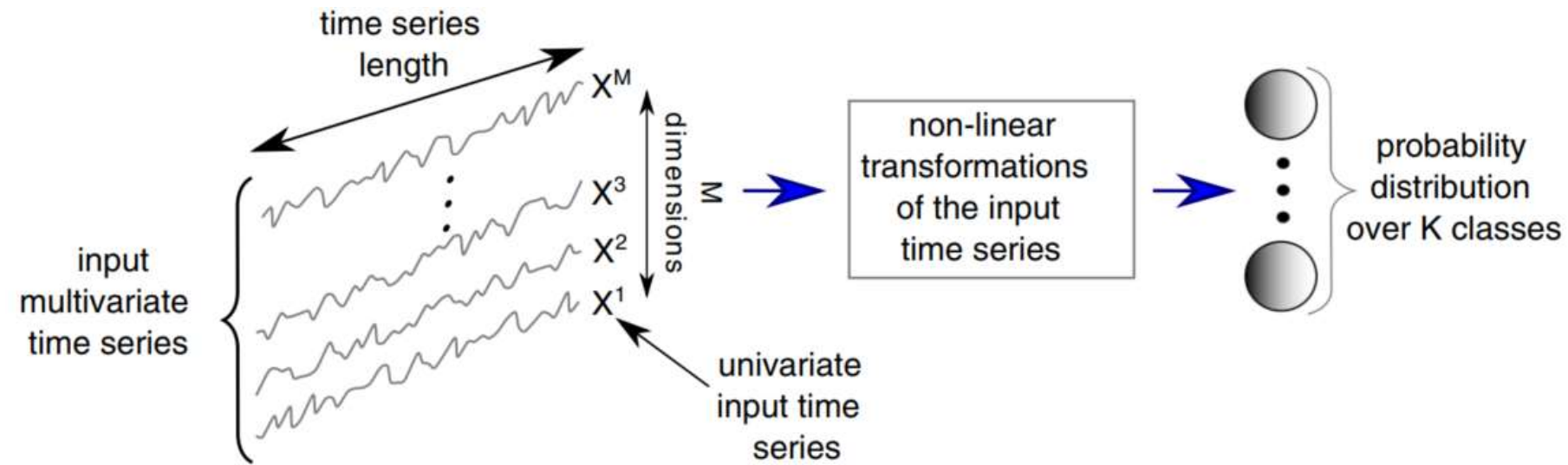
[[17.08945799]	[[16.775898]
[29.92123391]	[29.836689]
[-17.34433333]	[-14.146203]
[-0.2633606]	[-4.3511004]
[-15.84726045]	[-16.751232]
[15.49148883]	[17.285257]
[-4.55410355]	[-4.0891576]
[-4.52508266]	[-4.3335075]
[61.41969113]	[63.473457]
[1.56489795]	[-4.326838]

3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

- Данные функционирования высокотехнологичных информационных и инженерных систем, таких как МКА, на сегодняшний день по своим объемам и принципам работы с ними относятся к категории Big Data.
- Для решения задач анализа таких данных эффективным является разработка и применение методов и моделей машинного обучения с целью извлечения из них полезной информации и последующего построения с их использованием кластерно-классификационных и прогностических моделей для определения технического состояния МКА и принятия корректных управляющих и эксплуатационных решений МКА.



3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети



3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

- Исходные данные ТМИ являются временным рядом, который можно представить как матрицу $X = (x_{ij})$, где i -я строка X_i является анализируемым вектором показателей ТМИ в i -й момент времени, индекс j соответствует j -му показателю ТМИ в i -м векторе X_i .
- Данные ТМИ являются M -мерным временным рядом $X = (X_1, X_2, \dots, X_M)$, каждый элемент которого X_j является столбцом матрицы данных ТМИ X и в тоже время одномерным временным рядом, описывающим поведение j -го показателя ТМИ на отрезке дискретных моментов времени $[1, T]$.
- Для каждого вектора показателей ТМИ в i -й момент времени X_i в соответствие поставлена метка класса $y_i \in Y$, который характеризует состояние функционирования анализируемого по данным ТМИ МКА или его подсистемы.
- Рассматриваем случаи k -классовой классификации, где k – общее число состояний, определяемое экспертом. Конечной целью является классификация векторов X_i M -мерного временного ряда X ТМИ к 1 штатному и $k-1$ нештатным состояниям. В этом случае значения компонент вектора меток классов $Y \in \{0, 1, \dots, k-1\}$, где 0 обозначает штатное состояние и $1, \dots, k-1$ – нештатные состояния анализируемого МКА или его подсистемы.
- Стоит задача нахождения классификационной модели следующего нелинейного отображения:

$$y: X \rightarrow Y.$$

3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

Экспериментальные данные ТМИ МКА

- ❑ Для проведения компьютерных экспериментов и анализа точности разработанных нейросетевых классификационных моделей использованы данные ТМИ навигационной подсистемы МКА.
- ❑ Для БКА вектор матрицы ТМИ X_i имеет размерность 9 и помечается 0 в случае штатного состояния и 1 в случае нештатного состояния подсистемы. Общая размерность 9-мерного временного ряда X составляет 121690 векторов, из которых 77881 векторов составляет обучающий набор данных, 19471 векторов составляет валидационный набор данных, 24338 вектор составляет тестовый набор данных.

3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

Построим программную модель полносвязной нейронной сети:

```
n_class = 3

model = keras.models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(x.shape[1],)))
model.add(layers.Dense(16, activation='relu', kernel_regularizer='l1', bias_regularizer='l1'))
model.add(layers.Dense(16, activation='relu', kernel_regularizer='l1', bias_regularizer='l1'))
model.add(layers.Dense(n_class, activation='softmax'))

model.summary()
```

Model: "sequential_9"

Layer (type)	Output Shape	Param #
dense_36 (Dense)	(None, 16)	800
dense_37 (Dense)	(None, 16)	272
dense_38 (Dense)	(None, 16)	272
dense_39 (Dense)	(None, 3)	51

Total params: 1395 (5.45 KB)

Trainable params: 1395 (5.45 KB)

Non-trainable params: 0 (0.00 Byte)

3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

Компилируем модель полносвязной нейронной сети:

```
"compiling"  
model.compile(optimizer= 'adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

Обучаем с валидацией модель:

```
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor='val_loss',  
        patience=25,  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath='Dense_AIST_1_2.05.h5',  
        monitor='val_accuracy',  
        save_best_only=True,  
    ),  
    keras.callbacks.ReduceLROnPlateau(  
        monitor='val_loss',  
        factor=0.25,  
        patience=10,  
    )  
]  
  
start_time = time.time()  
m=model.fit(x_train, y_train, validation_data=(x_val, y_val), callbacks=callbacks_list, epochs=150, batch_size=128)  
  
duration = time.time() - start_time  
print('Duration time:', duration)
```

Epoch 149/150

14/14 [=====] - 0s 4ms/step - loss: 0.2097 - accuracy: 0.9936 - val_loss: 0.3602 - val_accuracy: 0.9

604 - lr: 0.0010

Epoch 150/150

14/14 [=====] - 0s 4ms/step - loss: 0.2075 - accuracy: 0.9924 - val_loss: 0.3572 - val_accuracy: 0.9

604 - lr: 0.0010

Duration time: 9.34998869895935

3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

Тестируем модель:

```
test_loss, test_acc = model.evaluate(x_test,y_test)
print('test_loss:', test_loss, ' ', 'test_acc:', test_acc)
```

Результаты обучения с валидацией и тестирования:

```
17/17 [=====] - 0s 1ms/step - loss: 0.3062 - accuracy: 0.9627
test_loss: 0.3062237501144409    test_acc: 0.9626865386962891
```

3.9.2 Классификация данных телеметрии МКА с помощью полносвязной нейронной сети

❑ Вывод графиков качества модели нейронной сети:

```
plt.plot(model.history.history['loss'], color="b", label='Train loss')
plt.plot(model.history.history['val_loss'], color="g", label='Valid loss')
plt.legend()
plt.show()

plt.plot(model.history.history['accuracy'], color="b", label='Train accuracy')
plt.plot(model.history.history['val_accuracy'], color="g", label='Valid accuracy')
plt.legend()
plt.show()
```

